IKARUS – Stage 2: Advanced Coding Challenge

Five Coding Problems with Sample Inputs, Expected Outputs, and Organizer Solutions

Total marks: 100 **Suggested duration:** 60 minutes

Problem 1: Smart Queue Scheduler

Area: Algorithmic Problem Solving **Language:** Python 3 **Marks:** 20

Problem statement: A service center receives customers with different priorities. Each customer record contains a token ID, arrival time, and priority. Arrange customers by higher priority first. If priorities are equal, arrange by earlier arrival time; if both are equal, use token ID in ascending order.

Input format: First line contains N. Each of the next N lines contains token ID, arrival time, and priority.

Sample input:

```
4
A 0 2
B 1 3
C 2 3
D 3 1
```

Expected output:

```
B C A D
```

Organizer solution (Python 3):

```
n = int(input())
customers = []

for _ in range(n):
    token, arrival, priority = input().split()
    customers.append((token, int(arrival), int(priority)))

customers.sort(key=lambda x: (-x[2], x[1], x[0]))
print(" ".join(c[0] for c in customers))
```

Problem 2: Duplicate Transaction Detector

Area: SQL Challenge **Language:** MySQL **Marks:** 15

Problem statement: A payment system must identify possible duplicate successful transactions. Two transactions are considered duplicates if they belong to the same user, have the same amount, both have status SUCCESS, and occur within 120 seconds of each other. Display the user ID and both transaction IDs.

Sample table: Transactions

transaction_id	user_id	amount	status	transaction_time
101	U1	500	SUCCESS	10:00:00
102	U1	500	SUCCESS	10:01:00
103	U2	300	SUCCESS	10:05:00
104	U1	500	FAILED	10:01:30

Expected output:

```
user_id  transaction_1  transaction_2
U1       101            102
```

Organizer solution (MySQL):

```
SELECT
    t1.user_id,
    t1.transaction_id AS transaction_1,
    t2.transaction_id AS transaction_2
FROM Transactions t1
```

```
JOIN Transactions t2
    ON t1.user_id = t2.user_id
   AND t1.amount = t2.amount
   AND t1.transaction_id < t2.transaction_id
   AND ABS(TIMESTAMPDIFF(
       SECOND, t1.transaction_time, t2.transaction_time
   )) <= 120
WHERE t1.status = 'SUCCESS'
  AND t2.status = 'SUCCESS'
ORDER BY t1.transaction_id, t2.transaction_id;
```

Problem 3: Clean and Analyze Student Data

Area: ML Preprocessing & Data Analysis **Language:** Java **Marks:** 20

Problem statement: A student analytics system receives student IDs and marks. Identify duplicate records, count records with invalid marks (less than 0 or greater than 100), and calculate the average marks using only unique valid records. If no valid records remain, display an appropriate message.

Input format: First line contains N. Each following line contains student ID and marks.

Sample input:

```
5
S01 80
S02 70
S01 80
S03 120
S04 90
```

Expected output:

```
Duplicate records: 1
Invalid records: 1
Average marks: 80.00
```

Organizer solution (Java):

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());

        Set<String> seen = new HashSet<>();
        int duplicates = 0, invalid = 0, count = 0;
        int total = 0;

        for (int i = 0; i < n; i++) {
            String id = sc.next();
            int marks = sc.nextInt();
            String record = id + " " + marks;

            if (seen.contains(record)) {
                duplicates++;
                continue;
            }
            seen.add(record);

            if (marks < 0 || marks > 100) {
                invalid++;
                continue;
            }

            total += marks;
            count++;
        }

        System.out.println("Duplicate records: " + duplicates);
        System.out.println("Invalid records: " + invalid);

        if (count > 0) {
            System.out.printf("Average marks: %.2f%n", (double) total / count);
        } else {
            System.out.println("Average marks: No valid data");
        }
    }
```

```
}
```

Problem 4: Debug the API Temperature Analyzer

Area: Debugging & API/Data Processing **Language:** Python 3 **Marks:** 20

Problem statement: An API returns a JSON array of city temperature records. Some records may have a missing temperature, null value, or non-numeric value. Process only numeric temperatures, count skipped records, and calculate the average of valid readings. The program must not crash when a record is incomplete.

Input: One line containing a JSON array.

```
[{"city":"Hyderabad","temp":30},{"city":"Delhi","temp":null},{"city":"Chennai","temp":32},{"city":"Mumbai"},{"city":"Pu
```

Expected output:

```
Valid readings: 2
Skipped records: 3
Average temperature: 31.00
```

Organizer solution (Python 3):

```
import json

data = json.loads(input())
total = 0
valid_count = 0
skipped = 0

for record in data:
    temp = record.get("temp")

    if isinstance(temp, (int, float)) and not isinstance(temp, bool):
        total += temp
        valid_count += 1
    else:
        skipped += 1

print("Valid readings:", valid_count)
print("Skipped records:", skipped)

if valid_count > 0:
    print(f"Average temperature: {total / valid_count:.2f}")
else:
    print("Average temperature: No valid data")
```

Problem 5: Optimize Resource Allocation

Area: Optimization Problem **Language:** Python 3 **Marks:** 25

Problem statement: A team has a limited time budget and several tasks. Each task has an ID, required time, and score. Select a subset of tasks that maximizes the total score without exceeding the time budget. Each task can be selected at most once.

Input format: First line contains N. Each of the next N lines contains task ID, time, and score. The final line contains the available time budget.

Sample input:

```
4
A 2 6
B 3 10
C 4 12
D 1 4
6
```

Expected output:

```
Selected tasks: A B D
Total time: 6
Total score: 20
```

Organizer solution (Python 3):

```
n = int(input())
tasks = []

for _ in range(n):
```

```
    task_id, time, score = input().split()
    tasks.append((task_id, int(time), int(score)))

capacity = int(input())
dp = [[0] * (capacity + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    task_id, time, score = tasks[i - 1]

    for c in range(capacity + 1):
        dp[i][c] = dp[i - 1][c]
        if time <= c:
            candidate = dp[i - 1][c - time] + score
            dp[i][c] = max(dp[i][c], candidate)

selected = []
c = capacity

for i in range(n, 0, -1):
    if dp[i][c] != dp[i - 1][c]:
        task_id, time, score = tasks[i - 1]
        selected.append(task_id)
        c -= time

selected.reverse()
total_time = sum(
    time for task_id, time, score in tasks
    if task_id in selected
)

print("Selected tasks:", " ".join(selected))
print("Total time:", total_time)
print("Total score:", dp[n][capacity])
```

Marks Distribution

Problem	Topic	Marks
1	Algorithmic Problem Solving	20
2	SQL Challenge	15
3	ML Preprocessing & Data Analysis	20
4	Debugging & API/Data Processing	20
5	Optimization	25
	Total	**100**